Listing 5.18 `tmdb.py` (top of file)

```
from semantic_kernel.functions import kernel_funct
import requests
import inspect


def print_function_call():     #1
    #omitted …


class TMDbService:     #2
    def __init__(self):
        # enter your TMDb API key here
        self.api_key = "your-TMDb-api-key"


    @kernel_function(     #2
        description="Gets the movie genre ID for a given genre name",
        name="get_movie_genre_id",
        input_description="The movie genre name of the genre_id to get",
        )
    def get_movie_genre_id(self, genre_name: str) -> str:     #3
        print_function_call()
        base_url = "https://api.themoviedb.org/3"
        endpoint = f"{base_url}/genre/movie/list↪
                     ↪ ?api_key={self.api_key}&language=en-US"

        response = requests.get(endpoint)     #4
        if response.status_code == 200:    #4
            genres = response.json()['genres']
            for genre in genres:
                if genre_name.lower() in genre['name'].lower():
                    return str(genre['id'])     #5
        return None
```

#1 Prints the calls to the functions for debugging
#2 Top-level service and decorator used to describe the function (good descriptions are important)
#3 Function wrapped in semantic wrapper; should return str
#4 Calls the API endpoint, and, if good (code 200), checks for matching genre
#5 Found the genre, returns the id

The bulk of the code for the `TMDbService` and the functions to call the TMDB endpoints was written with the help of GPT-4 Turbo. Then, each function was wrapped with the `sk_function` decorator to expose it semantically.

A few of the TMDB API calls have been mapped semantically. Listing 5.19 shows another example of a function exposed to the semantic service layer. This function pulls a current top 10 list of movies playing for a particular genre.